

Race-tracker radio protocol reference

Every frame type in `src/protocol.js`, byte-for-byte - sizes, field offsets, encodings, and real example frames generated from the actual encode/decode functions. For the transport this rides on (radios, wiring), see the separate rover wiring PDF.

General conventions

- All multi-byte integers are **little-endian**.
- Every frame starts with a one-byte **sync byte**, unique per frame type, and ends with a one-byte **checksum**: the sum of every byte from offset 1 (just after sync) through the byte before the checksum, mod 256. The sync byte itself is *not* included in the checksum.
- **boatId** is always 5 raw ASCII bytes (not null-terminated) - a fixed-width slot, not a C string.
- Latitude/longitude are **int32**, degrees $\times 1e7$ (about 1.1cm of resolution).
- Speed is **uint16**, knots $\times 10$. Heading is **uint16**, degrees $\times 10$ (0-3599, always normalized to 0-360° first).
- Six of the seven frame types are **fixed-length** - a receiver scanning the byte stream (`radioLink.js`) can look up the exact length from the sync byte alone. The batch frame (0xEE) is the one exception: its own second byte is a count that determines the rest of its length - see its own section for how a receiver handles that before the whole frame has even arrived.
- There is no acknowledgement frame anywhere in this protocol, for any frame type - every send is fire-and-forget over a simplex-feeling shared channel; confirmation (where it matters at all) comes from a side effect the sender can observe independently, never a dedicated reply.

Byte-stream framing: how a receiver tells frames apart

All frame types share one physical serial link with no gaps or delimiters between transmissions - a receiver just sees one continuous stream of bytes. `radioLink.js`'s scanner finds the earliest occurrence of any known sync byte in its buffer, then looks up how many total bytes that frame type needs (a fixed constant for six of the seven types, or a small header-read for the batch frame - see below). If fewer bytes have arrived so far than that, it waits for more. Once enough bytes are present, it verifies the checksum: a match means a valid frame, decoded and emitted; a mismatch means this wasn't a real frame after all (a sync byte value that happened to occur inside another frame's own data, or genuine radio corruption) - the scanner discards just the one sync byte and resumes scanning from the very next byte, rather than the whole buffer.

Sync	Name	Direction	Size	Purpose
0xAA	Position	Rover -> Base	26 B (fixed)	One GPS fix
0xBB	Marks	Base -> Rover	107 B (fixed, 8 marks)	Course + base address + regatta name
0xCC	Ping	Base -> Rover	6 B (fixed)	"Report your position now" (Ping fleet)
0xDD	Hello	Rover -> Base	7 B (fixed)	"I'm online", before any GPS fix exists
0xEE	Batch	Rover -> Base	27-84 B (variable)	1-4 fixes packed into one send
0xFF	Set-mark	Rover -> Base	17 B (fixed)	Set a course mark to this rover's position
0x88	Mark-log	Rover -> Base	27 B (fixed)	Annotate the base's own log with a note

0xAA · Position frame

rover->base

The core, highest-frequency frame - one rover's own GPS fix. Sent whenever the boat has moved $\geq$ TX_DISTANCE_M since its last transmitted fix (or on the TX_INTERVAL_S heartbeat for a stationary boat). This is also exactly what a boat replies with after a Ping frame - there's no separate "pong" shape.

sync 1B	boatId 5B	time 6B	lat 4B	lon 4B	speed 2B	hdg 2B	status 1B	sum 1B
------------	--------------	------------	-----------	-----------	-------------	-----------	--------------	-----------

Offset	Field	Type	Notes
0	sync	uint8	0xAA
1-5	boatId	5B ASCII	fixed-width, not null-terminated
6-9	unix time	uint32 LE	whole seconds
10-11	ms	uint16 LE	0-999 - milliseconds within that second, giving full-precision timestamps without a 64-bit field
12-15	lat	int32 LE	degrees $\times$ 1e7
16-19	lon	int32 LE	degrees $\times$ 1e7
20-21	speed	uint16 LE	knots $\times$ 10
22-23	heading	uint16 LE	degrees $\times$ 10
24	status	uint8	bit0 = gnssFixOk · bits1-2 = carrSoln (0=none,1=RTK float,2=RTK fixed) · bits3-7 = numSV (0-31 satellites)
25	checksum	uint8	sum of bytes 1-24 mod 256

Example (26 bytes) - lat 40.898118, lon -118.382154, 4.5kn, hdg 41.2deg, RTK fixed, 14 sv:

```
0 aa 41 42 43 44 45 74 2b 6c 67 29 01 bc 8e 60 18
```

```
16 1c 51 70 b9 2d 00 9c 01 75 82
```

Decodes to: {boatId: "ABCDE", timestamp: 1735142260297, lat: 40.898118, lon: -118.382154, speedKnots: 4.5, headingDeg: 41.2, gnssFixOk: true, carrSoln: 2, numSV: 14}

0xBB · Marks frame

base->rover

Broadcasts the whole course, the base's own network address, and the selected regatta's name - the only way a rover (no Redis access of its own) ever learns any of this. Sent on startup, whenever a new boat is heard, on a slow heartbeat (default 60s), and immediately after any mark is edited. Fixed-length today because MARK_NAMES (8 marks) is a compile-time constant - the length formula itself is generic and shifts automatically if that list ever grows.

sync 1B	8 marks 64B	base IP 4B	ports 4B	pin gate 1B	regatta 32B	sum 1B
------------	----------------	---------------	-------------	----------------	----------------	-----------

Offset	Field	Type	Notes
0	sync	uint8	0xBB

1-64	8 × {lat, lon}	8B each (int32 LE pair)	one pair per name in MARK_NAMES order: windwardGreen, windwardBlack, leewardGreen, leewardBlack, pin, committeeStart, committeeFinish, finish - same ×1e7 fixed-point as the position frame
65-68	base IP	4 × uint8	one octet each, e.g. 10.0.0.5 - all-zero (0.0.0.0) means "no address to publish"
69-70	base upload port	uint16 LE	the log-upload HTTP server's port
71-72	base admin port	uint16 LE	the base's own admin dashboard port
73	pin boundary gate	uint8	0/1 - if on, a rover derives the boundary's far endpoint itself from pin/committeeStart, fresh every time; no lat/lon of its own is transmitted for it
74-105	regatta name	32B ASCII	zero-padded/truncated; trailing NUL bytes trimmed on decode. Frame is only ever sent once a regatta is selected
106	checksum	uint8	sum of bytes 1-105 mod 256

Example (107 bytes) - 8 marks, base 10.0.0.5:8080 (admin 8092), pin boundary on, "Fall Regatta 2026":

```

0  bb 40 d8 60 18 40 a5 70 b9 50 ff 60 18 30 7e 70
16 b9 60 26 61 18 20 57 70 b9 70 4d 61 18 10 30 70
32 b9 80 74 61 18 00 09 70 b9 90 9b 61 18 f0 e1 6f
48 b9 a0 c2 61 18 e0 ba 6f b9 b0 e9 61 18 d0 93 6f
64 b9 0a 00 00 05 90 1f 9c 1f 01 46 61 6c 6c 20 52
80 65 67 61 74 74 61 20 32 30 32 36 00 00 00 00
96 00 00 00 00 00 00 00 00 00 00 00 00 bb

```

0xCC · Ping frame

base->rover

"Ping fleet" admin action - asks every rover to report its position right now, bypassing the normal movement gate. Built for a boat that's been sitting still since before the base came up: it already sent its one fix at that position and will never clear the distance gate again on its own. Carries no per-boat payload - every boat hears the same broadcast and replies independently (each after its own random jitter, so a whole fleet doesn't key up at once) with an ordinary position frame. There is no distinct "pong" shape.

sync 1B	unix time 4B	sum 1B
-------------------	------------------------	------------------

Offset	Field	Type	Notes
0	sync	uint8	0xCC
1-4	unix time	uint32 LE	whole seconds - when the ping was issued
5	checksum	uint8	sum of bytes 1-4 mod 256

Example (6 bytes):

0 cc 25 ec bb 6a 36

0xDD · Hello frame

rover->base

Sent unprompted, once, right at radio startup - before this rover even has a GPS fix to report. Lets the base's dashboard show "boat online, radio confirmed, still waiting on GPS" instead of nothing at all. Never written to CSV/Redis/RegattaUp - it only ever updates a live "last seen" indicator, nothing durable. The opposite direction and purpose from Ping above: this is the rover announcing itself, not the base asking for a report.

sync 1B	boatId 5B	sum 1B
-------------------	---------------------	------------------

Offset	Field	Type	Notes
0	sync	uint8	0xDD
1-5	boatId	5B ASCII	same fixed-width field as the position frame
6	checksum	uint8	sum of bytes 1-5 mod 256

Example (7 bytes) - boatId "ABCDE":

0 dd 41 42 43 44 45 4f

0xEE · Batch frame

rover->base

An optional, variable-length variant of the position frame: packs several consecutive fixes from the SAME boat into one transmission instead of sending each on its own, trading a little latency (fixes wait to fill a batch, though never longer than TX_INTERVAL_S) for fewer, larger over-the-air transmissions. Controlled by TX_BATCH_SIZE (default 4, this fleet's own radio hardware's actual RF payload ceiling - MAX_BATCH_COUNT). TX_BATCH_SIZE=1 skips this frame type entirely - every send goes out as a plain position frame instead.

Variable length - the only frame type here that isn't a fixed size. A receiver reads just the count byte (offset 1) first to compute the rest of the frame's length before it has the whole thing in hand, rather than guessing: $7 + \text{count} \times 19 + 1$ bytes. A count outside 1-4 is treated as corrupt (a tiny 8-byte placeholder length, just enough to resync past it) rather than trusted blindly - a false sync-byte match on random noise can never stall the scanner waiting on an implausibly large frame that will never arrive.

sync	count	boatId	fix 1	fix 2	... up to 4	sum
1B	1B	5B	19B	19B	19B ea.	1B

Offset	Field	Type	Notes
0	sync	uint8	0xEE
1	count	uint8	1-4 - how many fixes follow
2-6	boatId	5B ASCII	shared by every fix in this batch - a batch is always one boat's own consecutive fixes, never a mix
7-	count × fix record	19B each	oldest fix first, each record laid out exactly like the position frame's own bytes 6-24 (unix time, ms, lat, lon, speed, heading, status) - just without its own sync/boatId/checksum
last	checksum	uint8	sum of every preceding byte from offset 1 mod 256

Total size by count: 1→27B, 2→46B, 3→65B, 4→84B (the maximum - stays under this fleet's actual radios' read-only NP=100 RF payload limit).

Example, count=2 (46 bytes) - two consecutive fixes from boat ABCDE:

```
0  ee 02 41 42 43 44 45 74 2b 6c 67 29 01 bc 8e 60
16 18 1c 51 70 b9 2d 00 9c 01 75 74 2b 6c 67 8d 01
32 34 8f 60 18 a8 51 70 b9 2d 00 a4 01 75 28
```

0xFF · Set-mark frame

rover->base

An operator (or mark mode's own automatic movement gate) asking the base to set a specific course mark to this rover's own current GPS position - the field case this whole app exists for: an operator physically places a mark, then sets it from the rover's own touchscreen with no WiFi in reach at all. Rides the same radio a position fix already goes out on. No ack frame exists for this (or anything else in this protocol) - confirmation is the base's own re-broadcast of the updated Marks frame the instant it applies the change.

sync	boatId	mark idx	cont.	lat	lon	sum
1B	5B	1B	1B	4B	4B	1B

Offset	Field	Type	Notes
0	sync	uint8	0xFF
1-5	boatId	5B ASCII	
6	mark index	uint8	index into MARK_NAMES (0=windwardGreen ... 7=finish) - not a string, to keep this frame small
7	continuous	uint8 (0/1)	whether this update came from mark mode's own automatic gate (1) vs. a manual one-off markset tap (0) - purely informational; the base applies either identically. Lets the base's admin map warn an operator before overwriting a mark a live mark-mode rover is still actively driving
8-11	lat	int32 LE	degrees × 1e7
12-15	lon	int32 LE	degrees × 1e7
16	checksum	uint8	sum of bytes 1-15 mod 256

A decode is rejected (returns null) not just on checksum failure but also on a mark index past the end of MARK_NAMES, or a lat/lon outside real-world range ($\pm 90/\pm 180$) - the same bounds the base's own setMarkLocation enforces, so a decode failure reads the same way regardless of which check actually caught it.

Example (17 bytes) - boat ABCDE sets "pin" (index 4) to 40.898118, -118.382154, continuous=true:
0 ff 41 42 43 44 45 04 01 bc 8e 60 18 1c 51 70 b9
16 ac

0x88 · Mark-log frame

rover->base

An operator marking a moment in the base's own log with a short free-text note - built for field testing: walking/driving a rover to a new distance, checking an antenna, or any other physical change worth timestamping, so that moment lands inline in the base's own console and (when LOG_RATE_STATS=1) its rate-stats CSV, instead of cross-referencing two separate logs by wall-clock time afterward. No position of its own - just a labeled instant. No GPS fix required to send one at all.

sync	boatId	label	sum
1B	5B	20B	1B

Offset	Field	Type	Notes
0	sync	uint8	0x88

1-5	boatId	5B ASCII	
6-25	label	20B ASCII	zero-padded/truncated, same convention as the Marks frame's regatta-name field; trailing NULs trimmed on decode. Empty string is valid - a plain "mark this moment" with no note attached
26	checksum	uint8	sum of bytes 1-25 mod 256

Example (27 bytes) - boat ABCDE, label "1000m from base":

```

0  88 41 42 43 44 45 31 30 30 30 6d 20 66 72 6f 6d
16 20 62 61 73 65 00 00 00 00 00 0c

```

race-tracker/src/protocol.js - generated reference from the live encode/decode functions, not a hand-maintained spec.